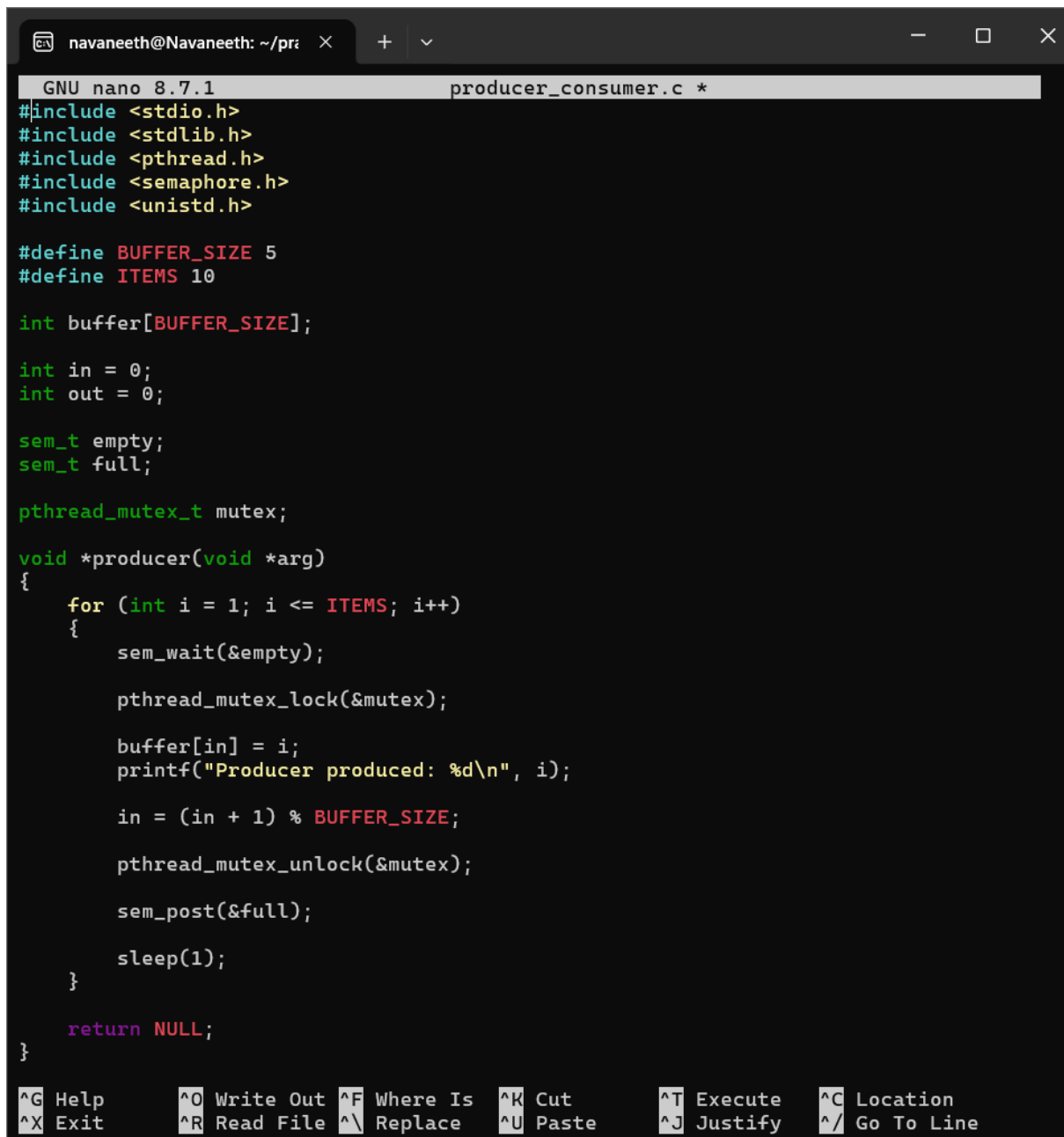


Practical Week 12

Aim

To implement the Producer–Consumer problem using POSIX threads and counting semaphores, and understand synchronization and buffer management.

Program:



```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Producer produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}
```

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

```
navaneeth@Navaneeth: ~/pr: x + v
GNU nano 8.7.1 producer_consumer.c *

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Output:

```
navaneeth@Navaneeth: ~/pr: x + v
navaneeth@Navaneeth:~$ mkdir -p ~/practical_week12
navaneeth@Navaneeth:~$ cd ~/practical_week12
navaneeth@Navaneeth:~/practical_week12$ nano producer_consumer.c
navaneeth@Navaneeth:~/practical_week12$ gcc producer_consumer.c -o producer_consumer -pthread
navaneeth@Navaneeth:~/practical_week12$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Producer produced: 3
Consumer consumed: 2
Producer produced: 4
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
Producer-Consumer execution completed.
navaneeth@Navaneeth:~/practical_week12$ |
```

Observation

The producer adds items to the shared buffer while the consumer removes them. Counting semaphores control the number of empty and full buffer positions, while the mutex protects the shared buffer from simultaneous access.

Result

The Producer-Consumer problem was successfully implemented using POSIX threads, counting semaphores, and mutex synchronization.

Aim

To create a deadlock scenario involving multiple threads competing for shared resources, identify the four necessary conditions for deadlock, and prevent deadlock using resource ordering.

Program:

```
navaneeth@Navaneeth: ~/pr: X + v - □
GNU nano 8.7.1 deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    printf("Thread 1 waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1 acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    sleep(1);

    printf("Thread 2 waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2 acquired Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-G Copy
```

```
navaneeth@Navaneeth:~/practical_week12$ nano deadlock.c
navaneeth@Navaneeth:~/practical_week12$ gcc deadlock.c -o deadlock -pthread
navaneeth@Navaneeth:~/practical_week12$ ./deadlock
Thread 2 locked Resource 2
Thread 1 locked Resource 1
Thread 2 waiting for Resource 1
Thread 1 waiting for Resource 2
^C
```

```
navaneeth@Navaneeth: ~/pr... x + v
GNU nano 8.7.1                                deadlock_prevention.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    printf("Thread 1 completed successfully\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    printf("Thread 2 completed successfully\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevention completed successfully.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

navaneeth@Navaneeth:~/practical_week12$ nano deadlock_prevention.c
navaneeth@Navaneeth:~/practical_week12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
navaneeth@Navaneeth:~/practical_week12$ ./deadlock_prevention
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 2 completed successfully
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 1 completed successfully
Deadlock prevention completed successfully.
navaneeth@Navaneeth:~/practical_week12$ |
```

Observation

The first program demonstrated a deadlock because the two threads acquired the resources in opposite orders. The second program acquired resources in the same predefined order, preventing circular waiting and allowing both threads to complete.

Result

A deadlock scenario was successfully demonstrated, the four necessary conditions were identified, and deadlock was prevented using resource ordering.